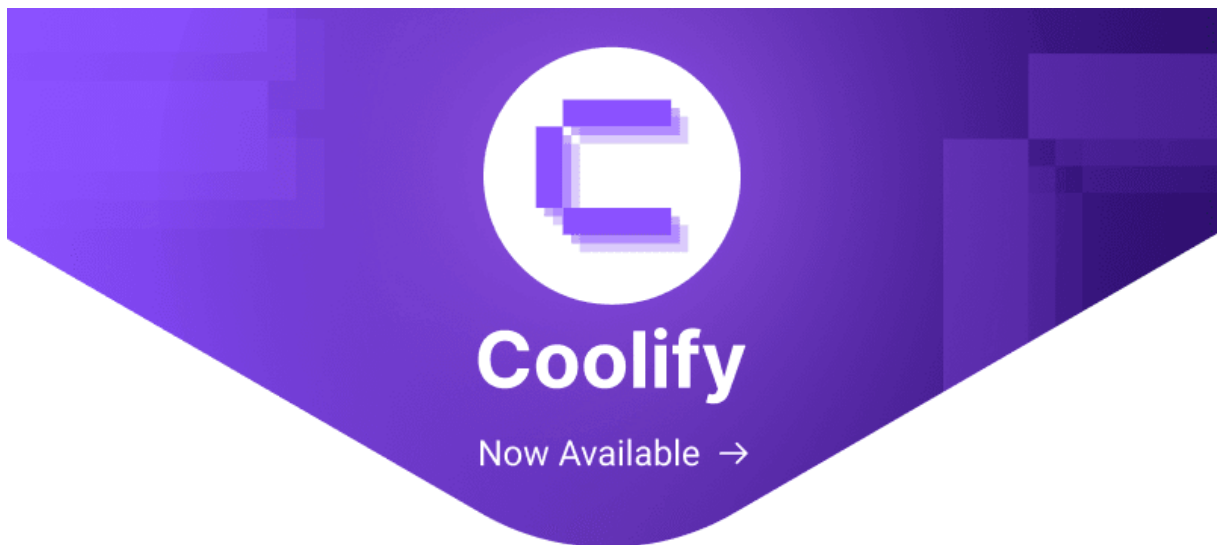


Coolify

Iván Alfaya Pérez

Diciembre 2025



 VULTR | Marketplace

Índice

1. Instalación y configuración de ngrok	3
1.1. Instalación	3
1.2. Configuración del token	3
2. Coolify	4
2.1. ¿Qué es Coolify?	4
2.2. Instalación de Coolify	5
2.3. Creación y despliegue de un proyecto	6
2.4. Base de datos	7
2.5. Pruebas de acceso a la API	9
3. Acceso externo a la API con ngrok	11
4. Webhooks	11
4.1. ¿Qué es un webhook?	12

[volver](#)

En este documento se describe el proceso completo de instalación y configuración de **Coolify**, la exposición de servicios mediante **ngrok** y la implementación de **webhooks** de **GitHub** para automatizar despliegues.

Comenzamos con la instalación de Coolify utilizando el siguiente comando:

Código

```
curl -fsSL [https://cdn.coollabs.io/coolify/install.sh]
(https://cdn.coollabs.io/coolify/install.sh) | bash
```

Además, utilizaremos **ngrok** para exponer servicios locales a Internet durante la fase de pruebas.

Enlace oficial para descargar ngrok en Linux:(seguir los pasos que te ponen, los vamos hacer nosotros también)

[Descargar ngrok para Linux](#)

1. Instalación y configuración de ngrok

[volver](#)

1.1. Instalación

[volver](#)

Ejecutamos los siguientes comandos para instalar ngrok en el sistema:

Código

```
curl -sSL [https://ngrok-agent.s3.amazonaws.com/ngrok.asc](https://ngrok-agent.s3.amazonaws.com/ngrok.asc) | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc
>/dev/null echo "deb [https://ngrok-agent.s3.amazonaws.com](https://ngrok-agent.s3.amazonaws.com) bookworm main" | sudo tee
/etc/apt/sources.list.d/ngrok.list sudo apt update sudo apt install ngrok
```

1.2. Configuración del token

[volver](#)

Para habilitar todas las funcionalidades de ngrok es necesario añadir nuestro **auth-token**, el cual vincula la herramienta con nuestra cuenta personal.

Este token se obtiene tras iniciar sesión en la web de ngrok.

Enlace: [inciar sesion](#)

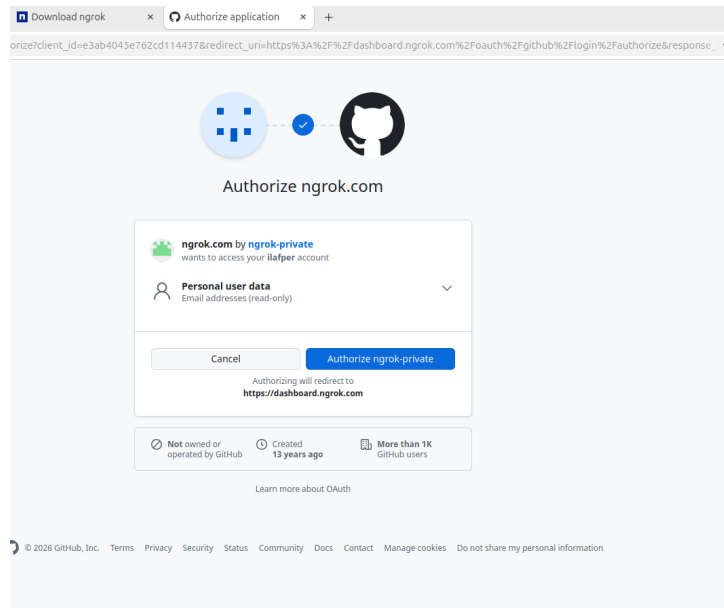


Figura 1: Obtención del token de autenticación en ngrok

Una vez copiado el token, lo añadimos en la terminal:

Código

```
ngrok config add-authtoken 38c9QmlPqqVJBvK7eYWHAmcO9h6FJHu657NyTMq8LogjfgB
```

A continuación, iniciamos un túnel HTTP:

Código

```
ngrok http 8000
```

Esto nos permitirá exponer nuestro servidor local mediante una URL pública, útil durante la fase de desarrollo y pruebas.

2. Coolify

[volver](#)

2.1. ¿Qué es Coolify?

[volver](#)

Coolify es una plataforma de despliegue y gestión de aplicaciones similar a servicios como Vercel o Render, con la diferencia de que es **autoalojada**. Esto significa que el usuario mantiene el control total sobre el servidor, las aplicaciones y las bases de datos.

Coolify automatiza despliegues desde repositorios Git, gestiona contenedores Docker, puertos y certificados HTTPS.

2.2. Instalación de Coolify

[volver](#)

Accedemos a la página oficial:

[\[https://coolify.io/\]](https://coolify.io/)(https://coolify.io/)

Seleccionamos la opción **Self-Hosted** y ejecutamos el comando proporcionado:

Código

```
curl -fsSL [https://cdn.coollabs.io/coolify/install.sh](https://cdn.coollabs.io/coolify/install.sh) | sudo bash
```

Una vez finalizada la instalación, accedemos desde el navegador a:

[\[http://localhost:8000\]](http://localhost:8000)(http://localhost:8000)

Creamos una cuenta rellenando el formulario correspondiente y luego iniciaremos sesión.

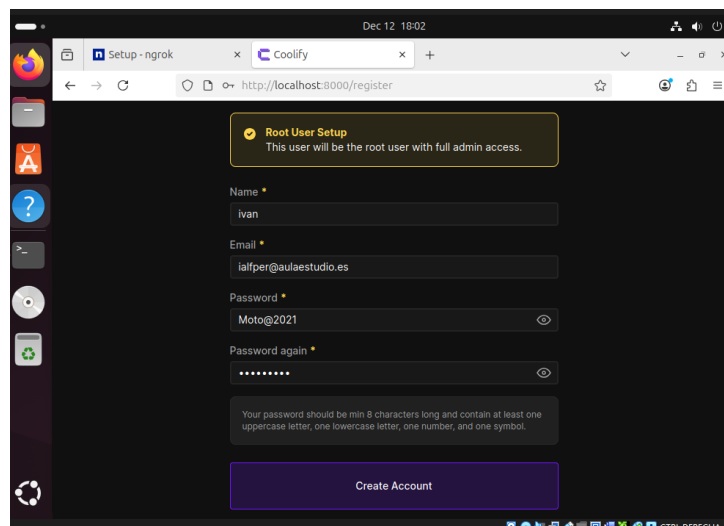


Figura 2: Formulario de creación de cuenta en Coolify

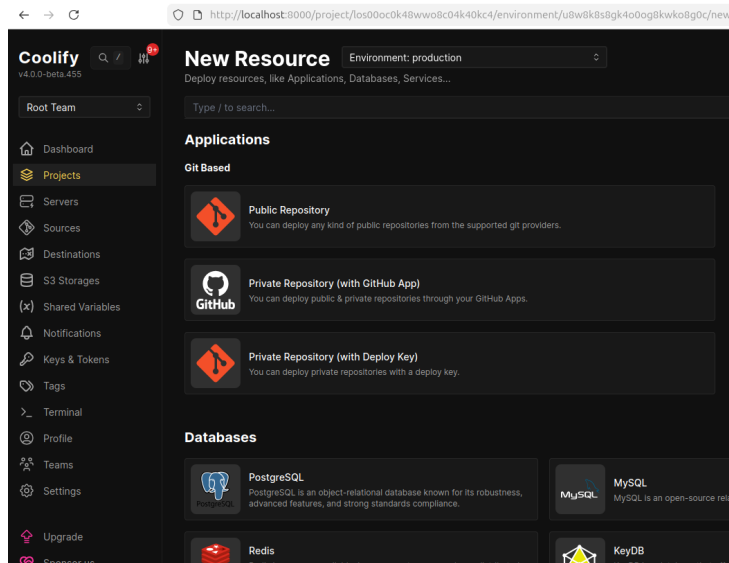


Figura 3: Pantalla principal de Coolify

2.3. Creación y despliegue de un proyecto

volver

Para el despliegue utilizaremos un repositorio público de GitHub:

[\[https://github.com/ilafper/Docker_Robert_api\]](https://github.com/ilafper/Docker_Robert_api)(https://github.com/ilafper/Docker_Robert_api)

Desde la pantalla principal de Coolify seleccionamos **Public Repository** e introducimos la URL del repositorio.

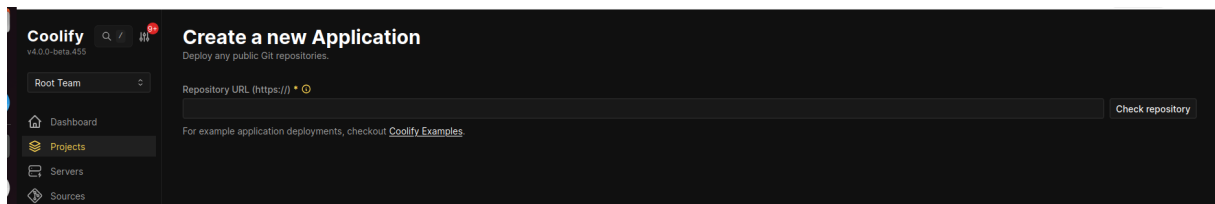


Figura 4: Enlace del repositorio en Coolify

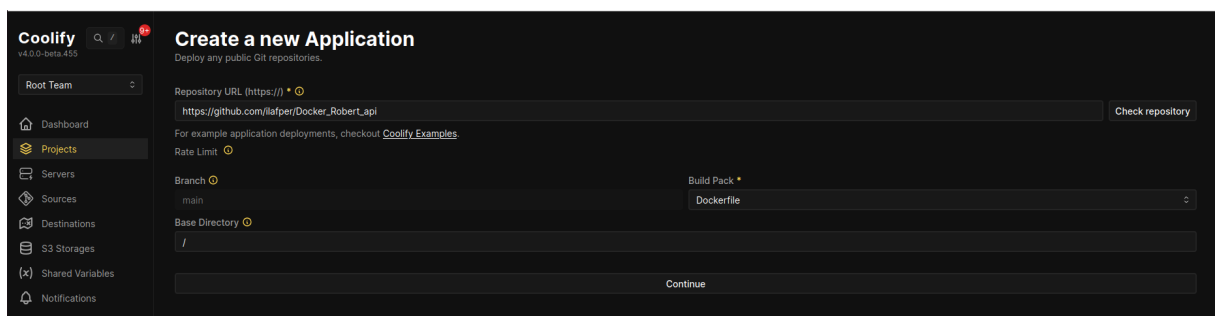


Figura 5: Configuración inicial del repositorio

En la sección **Build** dejamos el directorio base en la raíz y la localización del Dockerfile como `/Dockerfile`.

Guardamos los cambios y desplegamos el proyecto. El estado del despliegue puede verse en los logs:

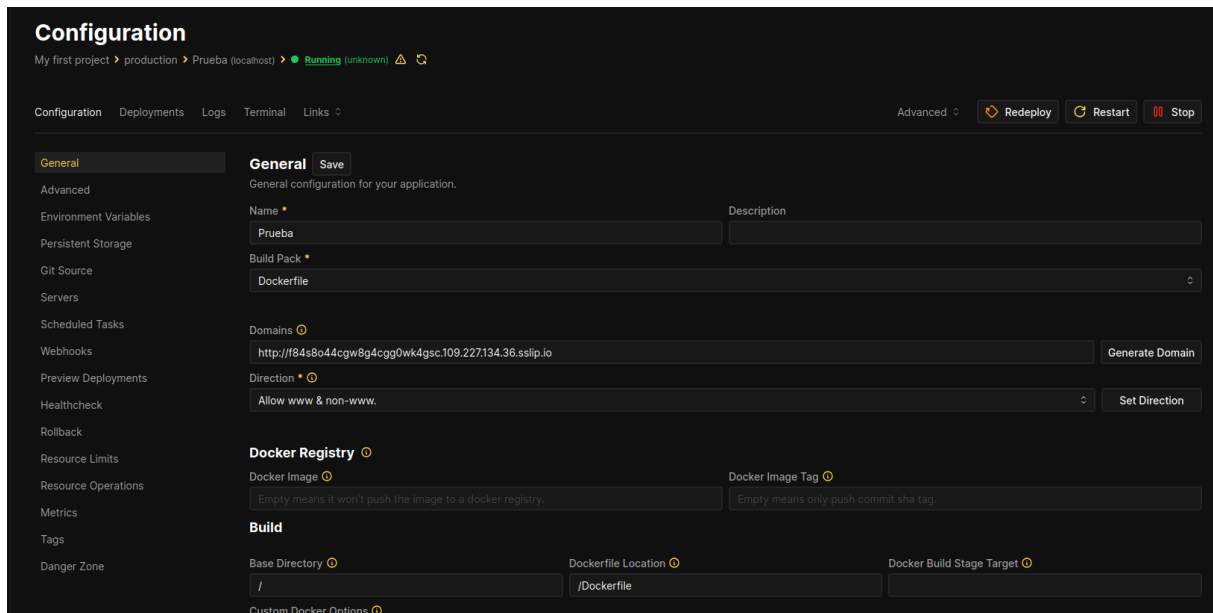


Figura 6: configuración proyecto

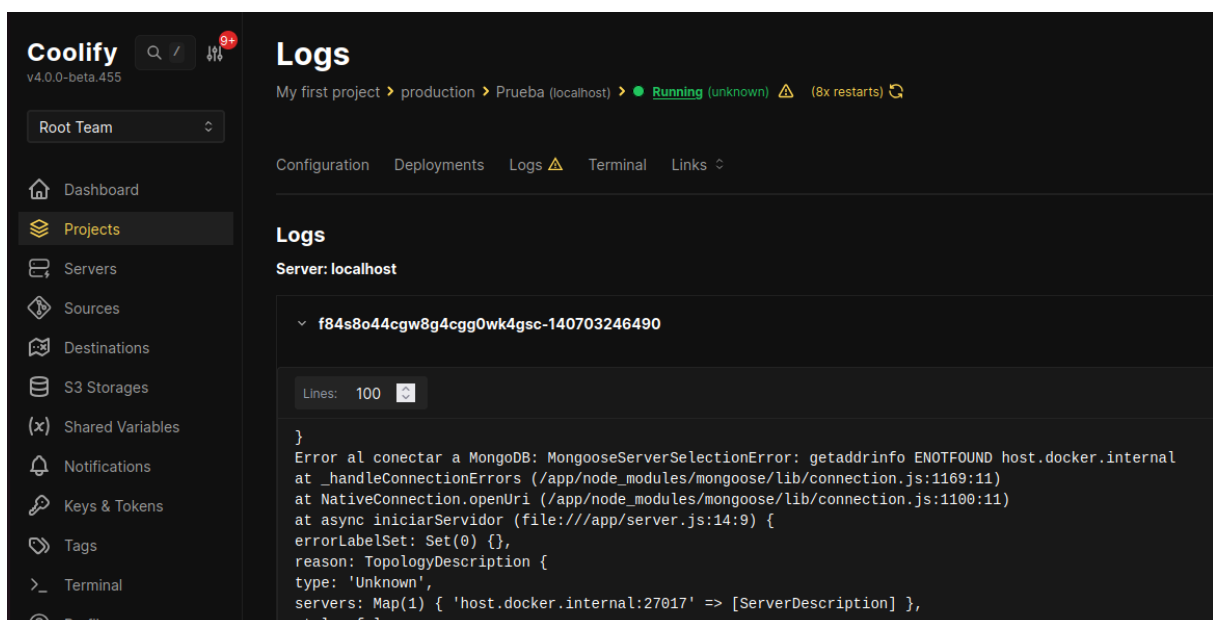


Figura 7: Logs del despliegue en Coolify

2.4. Base de datos

volver

El proyecto requiere una base de datos. En este caso utilizamos **MongoDB**.

Desde **Projects** → **Add Resource** creamos el recurso de base de datos y añadimos la variable de entorno correspondiente en el proyecto.

Esta variable de entorno va en el proyecto en enviroments, add y pones su nombre y valor, guardas

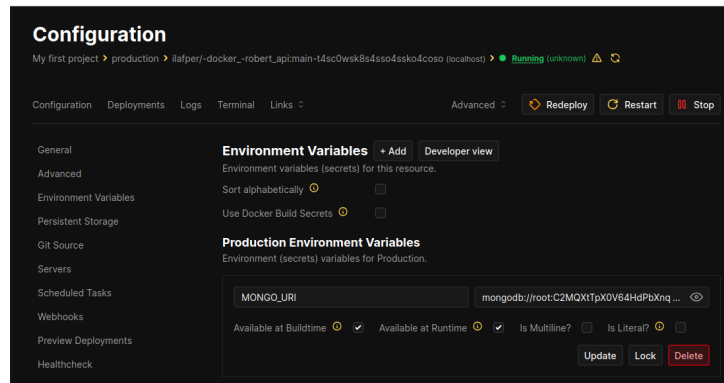


Figura 8: Variable de entorno para la base de datos

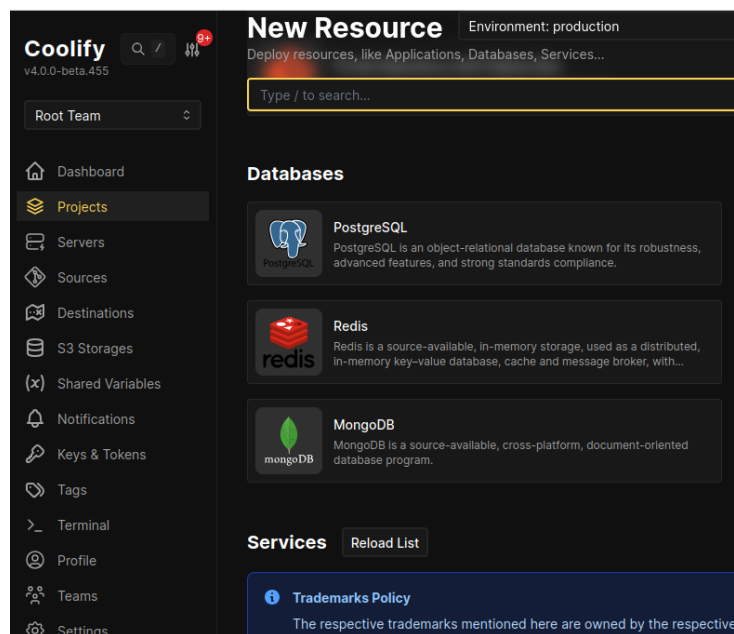


Figura 9: Creación del recurso de base de datos

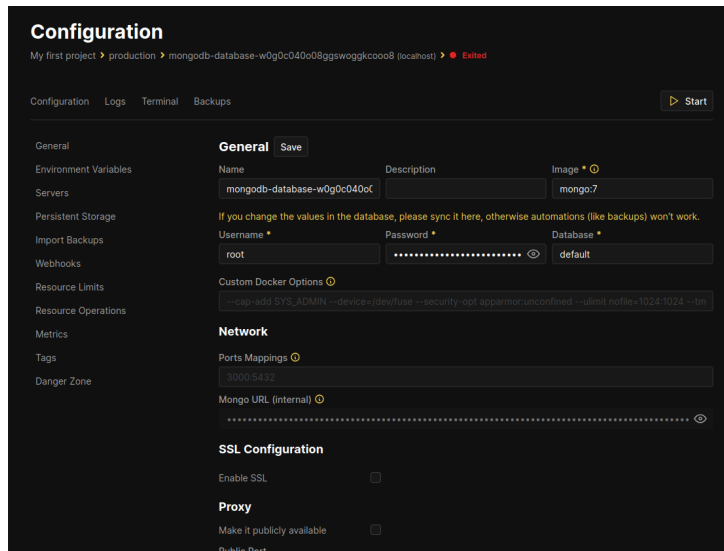


Figura 10: Parámetros de la base de datos

Iniciamos la base de datos y re desplegamos el proyecto.

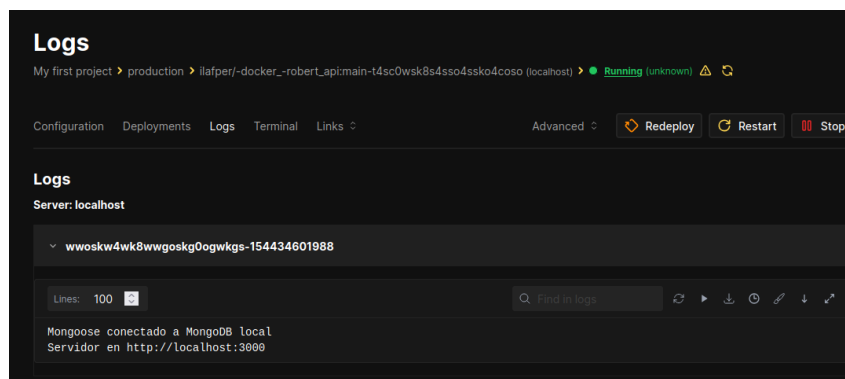


Figura 11: Proyecto conectado correctamente a la base de datos

2.5. Pruebas de acceso a la API

[volver](#)

Para conocer la IP de la máquina virtual instalamos las herramientas de red:

Código

```
sudo apt install net-tools
```

Código

```
ifconfig
```

Sustituimos la IP del dominio por la IP de la interfaz `enp0s3`, guardamos los cambios y redesplegamos.

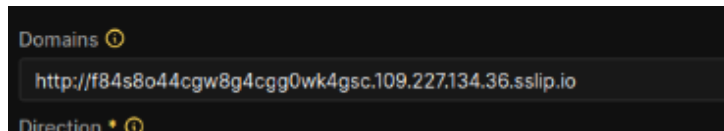


Figura 12: dominio Proyecto

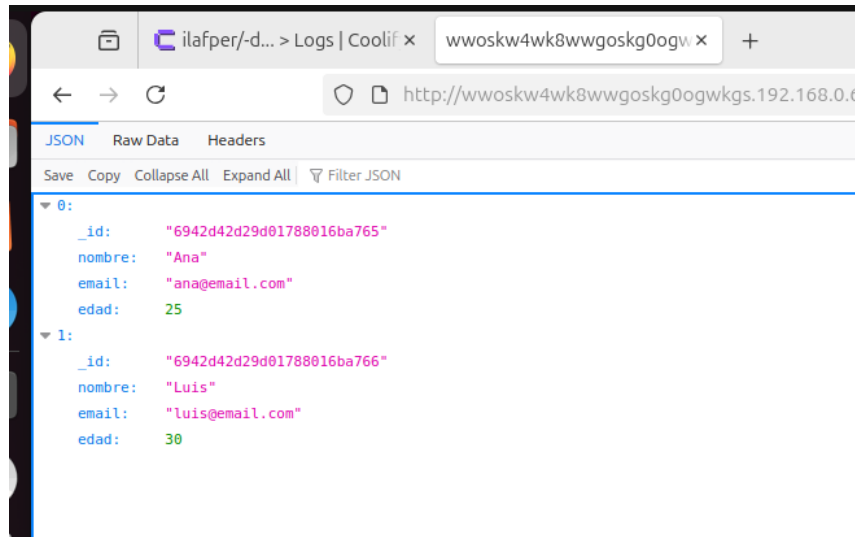


Figura 13: Prueba de la API en la máquina virtual

Desde la máquina anfitriona utilizamos el túnel de ngrok(no cerrar el tunel por que si no, no va):

Código

```
ngrok http 8000
```

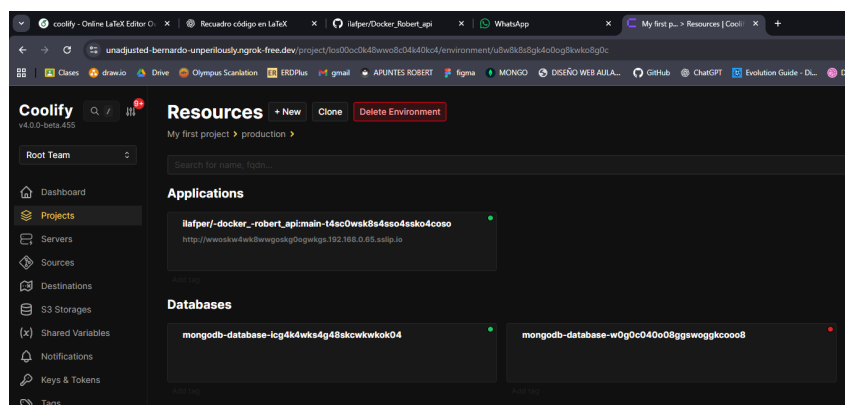


Figura 14: Acceso al proyecto desde la máquina anfitriona

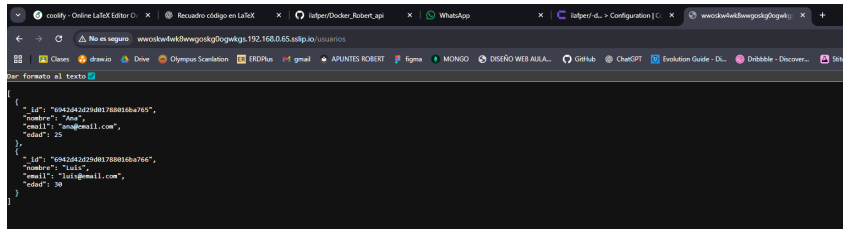


Figura 15: Prueba de la API desde el host

3. Acceso externo a la API con ngrok

[volver](#)

Para exponer el proyecto fuera de la red local utilizamos el siguiente comando:

Código

```
ngrok http 80 -host-header=wwoskw4wk8wwgoskg0ogwkgs.192.168.0.65.sslip.io
```

Explicación del comando

- `ngrok http 80`: crea un túnel HTTP al puerto 80.
- `-host-header`: redirige las peticiones al host interno del contenedor Docker.

Copiamos la URL pública generada y accedemos desde un dispositivo externo.

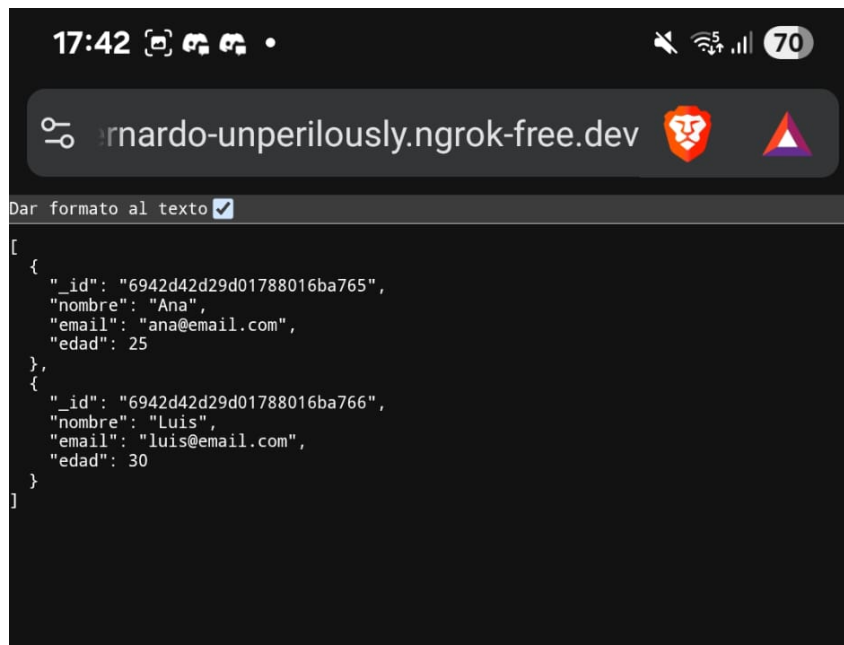


Figura 16: Prueba de la API desde un dispositivo móvil

4. Webhooks

[volver](#)

Los webhooks permiten recibir notificaciones automáticas desde GitHub cuando ocurre un evento en el repositorio, como un *push*.

4.1. ¿Qué es un webhook?

Un webhook es un mecanismo mediante el cual una aplicación envía peticiones HTTP en tiempo real a otra cuando ocurre un evento determinado.

Para generar un secreto podemos ir este enlace [generar Secreto](#)

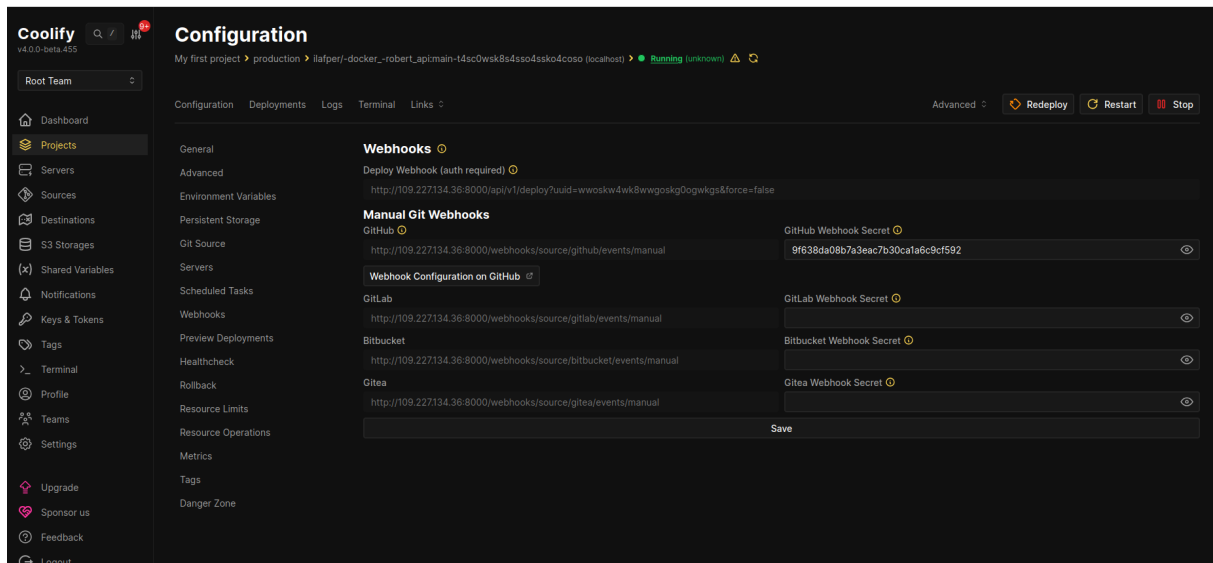


Figura 17: Configuración del webhook en Coolify

Ahora nos vamos al repositorio de nuestro proyecto, **Settings** → **Webhooks** → **add webhook** y completamos los campos.

En la url ponemos la siguiente: [\[https://unadjusted-bernardo-unperilously.ngrok-free.dev/webhooks/source/github/events/manual\]](https://unadjusted-bernardo-unperilously.ngrok-free.dev/webhooks/source/github/events/manual)(<https://unadjusted-bernardo-unperilously.ngrok-free.dev/webhooks/source/github/events/manual>)

Cuando creas el webhook tenemos que poner la url del servidor que recibira el aviso.

La url se forma con dos partes la primera es la url de cuando creas el tunel de ngrok (ngrok http 8000) y la otra esta parte del webhook de tu proyecto.

```
vboxuser@afundamentoUbuntu: ~/Desktop
ngrok
One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             ialfper@aulaestudio.es (Plan: Free)
Version             3.34.1
Region              Europe (eu)
Web Interface        http://127.0.0.1:4040
Forwarding            https://unadjusted-bernardo-unperilously.ngrok-free.dev -> http://localhost:80

Connections          ttl    opn    rt1    rt5    p50    p90
                     0      0      0.00   0.00   0.00   0.00
```

Figura 18: parte 1 url

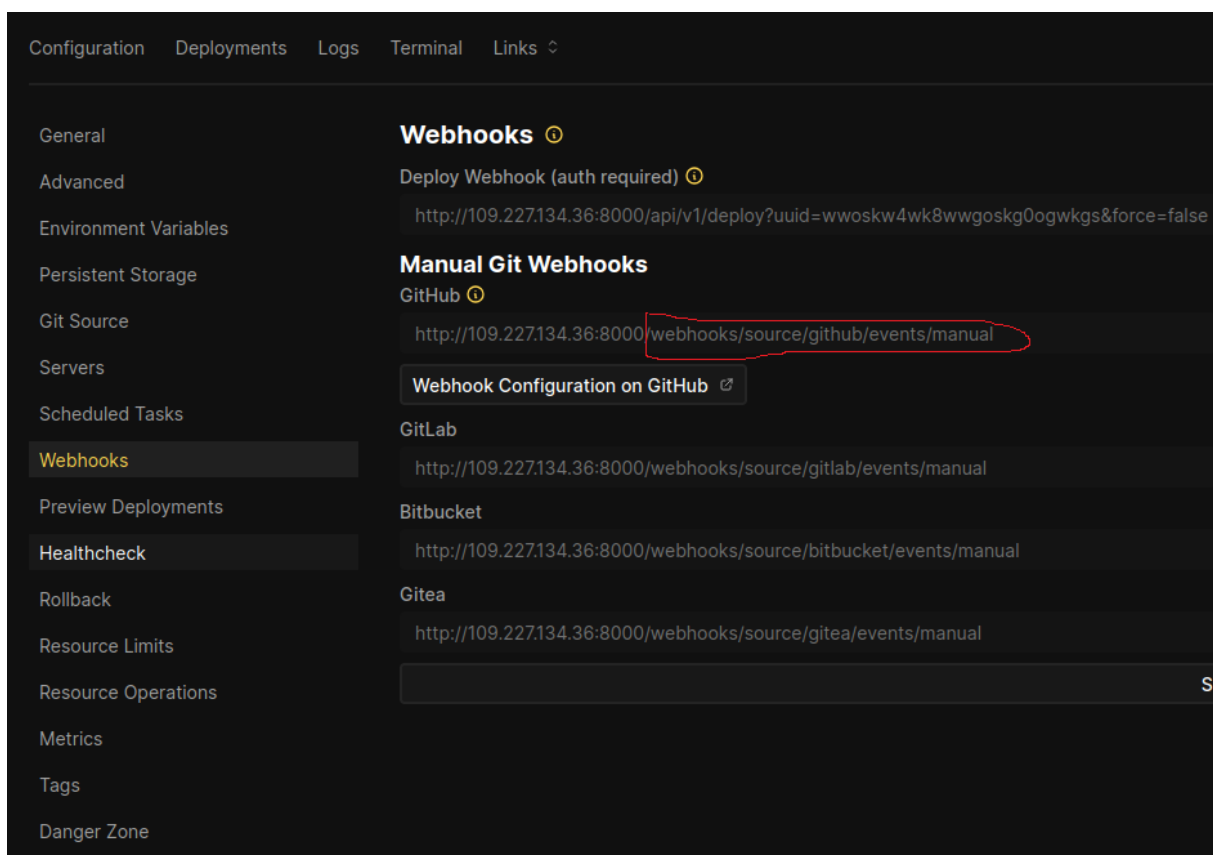


Figura 19: parte dos url

Y en el secret el mismo que añadimos al proyecto, el que generamos anteriormente.

Figura 20: Creacion webhook

El resto de las opciones como se ve en la imagen, que son la predefinidas.

Figura 21: Webhook creado correctamente

Realizamos un pequeño cambio en la API y hacemos *push* (en mi caso le añadi que en la raiz tenga un mensaje al principio por ejemplo):

```
app.get("/", async (req, res) => {
  res.json({ message: 'prueba webhook6' });
});
```

Hacemos un push de nuestro cambio en proyecto y volvemos al repositorio para comprobar

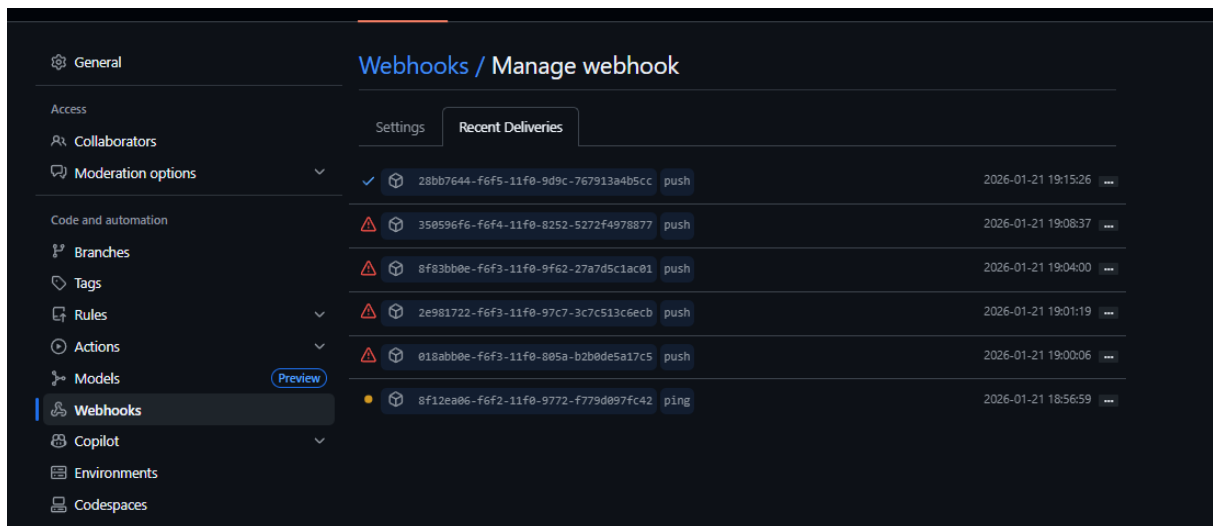


Figura 22: Push correcto detectado por el webhook

1. Revisa el estado del despliegue:

- Accede al webhook del repositorio picha en el y puedes ver si fue correcto o no.
- Haz clic en **Recent Deliveries**.
- Verifica si el proceso fue exitoso o si presenta algún error específico (ejemplo: error **404**).

2. Si el despliegue fue exitoso:

- Copia el dominio asignado a tu proyecto en Colify.
- Ábrelo en tu navegador web (o recarga la página si ya la tenías abierta).
- **Espera unos segundos**; los cambios no son instantáneos debido al caché y a la propagación.
- Recarga la página y visualiza tus cambios.

Nota Importante

No es necesario volver a desplegar manualmente. Con cada push a tu repositorio, el webhook activará automáticamente un nuevo despliegue en Colify.

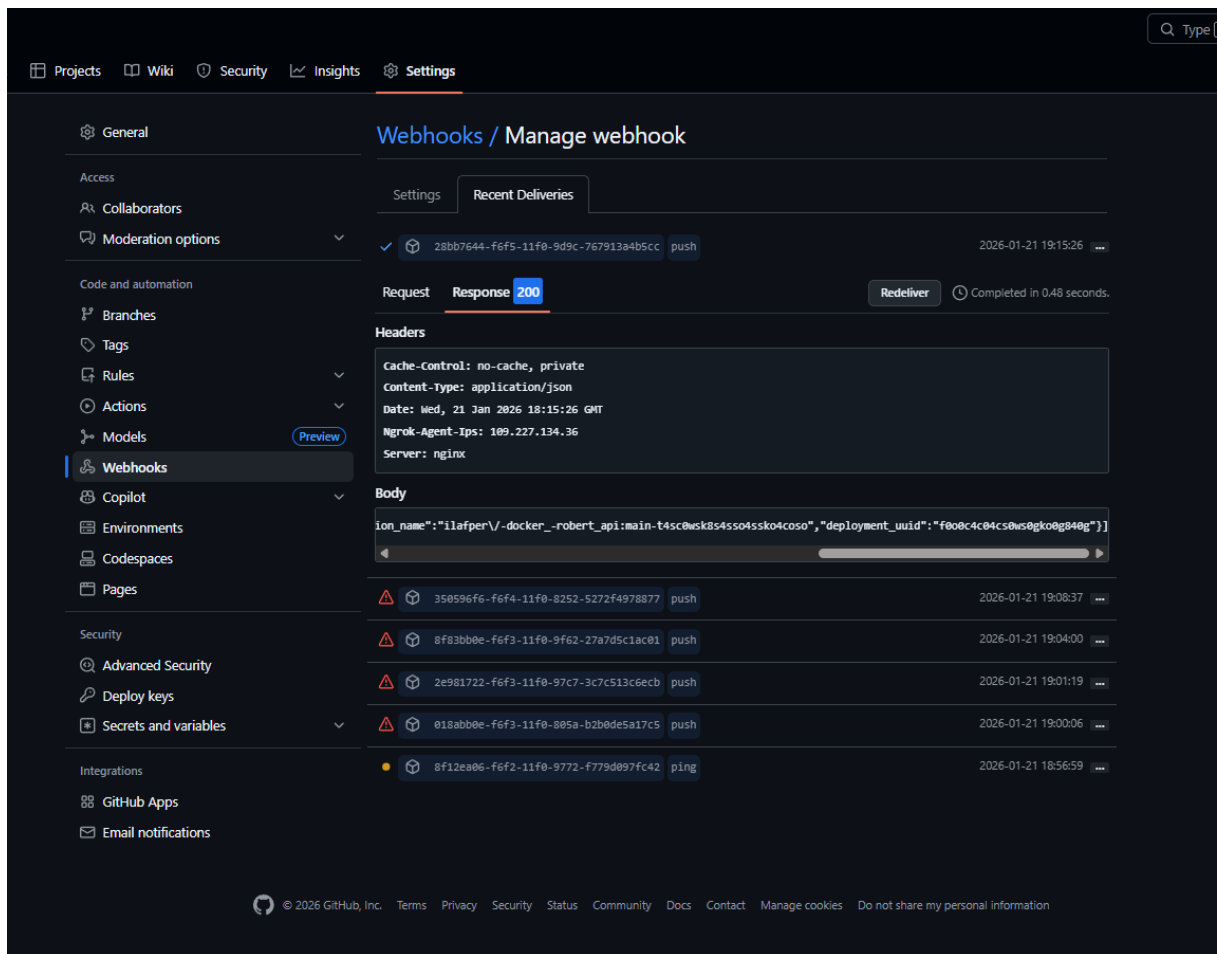


Figura 23: Webhook funcionando correctamente

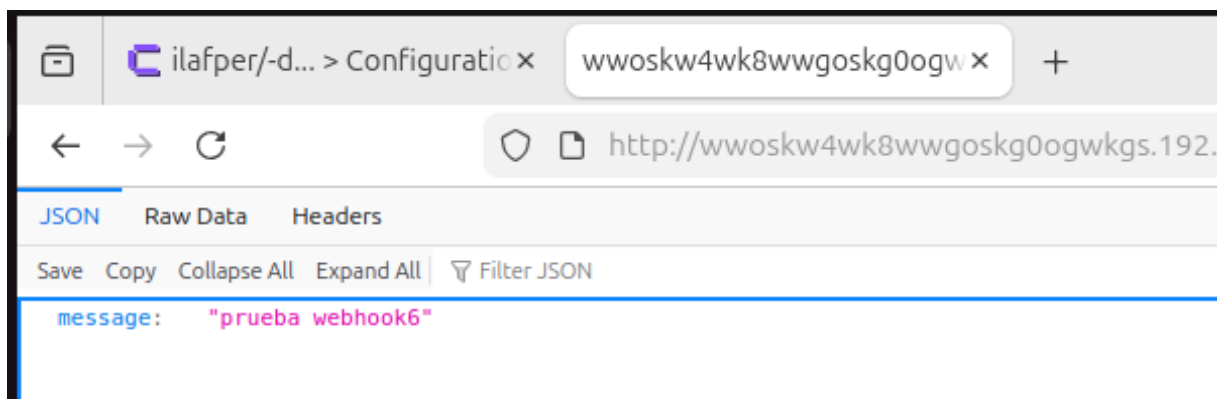


Figura 24: Resultado final del despliegue automático